

Note: The assignment 3 page says “max 3 pages” for the readme. However, I talked with Rahul directly and he said the more detail, the better, and I could go upwards of 5 pages if needed. Most of pages 5 and 6 are blank.

How to run the game

To begin a game, press “Play” and choose any save slot. They will auto-fill with default data in the background; save slots can be saved at any time during gameplay and will save the equipped weapon, player’s loot, dialogues played, audio group levels, weapon purchases and unlocks, the active scene, and the furthest beaten level. Once a slot is saved, it can be re-chosen through the main menu.

Optionally, change the audio values in the “Options” menu, or read the game explanation. You can still change the audio during the game in the pause menu’s option settings, but the game explanation can only be read in the main menu.

The game will explain to you how to play through dialogue we added, as well as give you some context around what is happening. For a high-level overview of the game, your goal is to rebuild some sort of renewable energy or pollutant cleaner in each level. The levels will all tell you how many components you need to pick up, and you will need to search around the map to find them.

Once a level is beaten, it will be permanently “cleaned.” The map will change to reflect that. *Note: the 3D level does not visually change on completion.*

Controls

Left click — Attack with equipped weapon in direction of movement, **Right click** — Secondary special attack with equipped weapon, **“E”** — interact when touching something, **WASD** — Movement, **Escape** — Open and close the pause menu, **Shift** — Dash

Where data lives

Our tuned variants (easy/hard) can be found under **Assets/Scripts/TUNED_VARIANTS/***. There are variants for player movement (2D and 3D, as they use different movement systems), player health, and enemies (including health, movement, attack rate).

However, we did not only use data as tuned variants. I separated ScriptableObjects into their distinct categories in the CreateAsset menu, so if you like, you can right click anywhere in the Assets folder, go to **Create**, and view the four distinct categories: **EntityData**, **EventObjects**, **DialogueObjects**, and **ScriptableObjects**.

- **EntityData**
 - These are the mentioned variants above—Health, Movement (2D/3D), EnemyData—associated with “entities.”
- **EventObjects**
 - These are data objects that work as wrappers for actions, creating “event channels” / “event streams.” I created multiple types for different sorts of events, and it makes it a lot easier to sync data when changing scenes, since certain objects persist between scenes and others don’t. If we didn’t use these EventObjects, we would have to re-subscribe to actions whenever a scene changed, which makes for some messy references. This reduces coupling between components significantly and honestly just makes everything a lot cleaner, at the cost of a little bit of extra setup time in the Inspector. EventObjects “live” scattered throughout the Scripts folder, close to whatever script should own them (so for instance, some **PurchaseEventObjects** live under **Assets/Scripts/Player/Looting and Currency**, as the PlayerLooter uses them).
- **DialogueObjects**
 - Dialogue is a feature I added for assignment 3, and it runs on three main ScriptableObjects: **DialogueBoxScripts**, **DialogueScenes**, and **DialogueEventTriggers**. **Scripts** hold text for one “box.” **Scenes** are built of a list of Scripts, and have a specific key associated with them so the save system can save when dialogue has been played and not replay it. **DialogueEventTriggers** hold a **Scene** and some **EventObject** trigger stream. This allows us to create dialogue for many different conditions, as we can link any **EventObject** to the **DialogueScene** through a **DialogueEventTrigger**.
 - All of our **DialogueObjects** can be found under **Assets/Scripts/Dialogue/Scenes/***.
- **ScriptableObjects**

- These are data objects that didn't fit into any of the above. There are only two, **WeaponData** and **LootObject**. **WeaponData** holds information about weapon cooldowns, and **LootObject** holds information about a loot drop, like frequency of common, rare, and mythic currency.

2D Improvements

Game feel & feedback: We added a flash screen that plays when the rebuildable is done, as well as glowing particles that play on pickup, as well as dust particles that play behind the player when they walk.

Balancing: Exposed many values through ScriptableObjects, like player movement, player health, enemy health, enemy attack data, weapon data, loot tables. Specifically to address the requirement of “Tuned variants” we created multiple versions of player movement, health, and enemy data that can be found under **Assets/Scripts/TUNED_VARIANTS/***.

HUD: We added a “component pickup” UI that effectively works as an inventory display. It runs on an event stream that passes item data through, and will render items by a 2D texture on the item prefabs themselves. When items are used by the player, it erases them from the visual inventory by matching their item ID.

Performance: We improved the pooling system for enemies, added a pool for loot objects, dust particles, dash particles, and sparkle particles. We removed all of the per-frame allocations that we were using with **Physics2D**. We did *not* disable unused components when offscreen, as I did research and determined that turning on / off components like the Animator or Colliders whenever they leave your render view is actually not as performant as setting Animators to Cull or Colliders to static. We set all objects to static whenever we could and made textures GPU instanced, which improved our render times.

General improvements: We completely reworked the way the save system works and set the Player to `DontDestroyOnLoad`, allowing us to reduce auto-save and load calls that were running on every scene change before (to sync data). By making the player `DontDestroyOnLoad`, we could remove the `ActiveGameManager` component we had before, merging it into the `SaveManager`, which we renamed to `GameManager`. `GameManager` now functions as the authoritative source of truth when it comes to game progression, and that allows us to much more easily switch between save slots. In general, we reduced the coupling between distinct components by switching to event streams instead of explicit **event Action** references, which allowed us to reduce the number of Singleton instances we had in the scene at any one time and made everything a lot more independent from one another.

Accessibility and Usability improvements: I go over the changes suggested by Rahul under the feedback changes, but in general, we improved the contrast of text, made font sizes larger, and gave the player more clear directions about what to do. Importantly, the font size on the WebGL build specifically tends to be smaller across the board, as does the UI. This bug is marked under “Known Issues.”

Feedback changes: see “Part 1 Feedback Addressed” further in the document.

3D Menu additions

3D Scene: We have a hand-made terrain with 3d objects scattered around it, depicting a sandy, destroyed terrain.

Camera: The camera has a short idle orbit around the scene showing off the environment.

UI: The menu buttons are in screen space and have click/hover feedback. There is a button to “Play” and choose a save slot (which leads to the Home Area, our interactive level select). There is also an options page, which lets the user lower different volume groups. There is a game explanation page detailing information about the game. There is a quit game button, but note that **this button uses `Application.Quit()` to function. On [Itch.io](https://itch.io), all this button will do is permanently freeze the game until you refresh**. I assume that's because the program runs in some container on the server and the process gets terminated (as it should) when you press the quit button.

Polish: We have a colored sunlight that gives the desert a more orange appearance. We also added fog, to make it seem like there's pollution in the air, and changed the skybox to fit with the desert theme better. Objects in the scene all have materials and when possible we set them to GPU instancing for performance improvements.

3D Level Additions

Mechanics: I gave the existing weapons 3d implementations (ranged and melee) and the goal of the 3d level is, like the prior scenes, to rebuild a collection of objects (in this case “mechanical trees” that sequester carbon dioxide from the atmosphere).

Core: We have a 3D character (without a model, as we could not find one we felt looked good with the scenario, and I didn’t have time to model and rig an entire character, especially as I have never really modeled or animated anything outside of this class). The character moves like in the 2D game (including the dash), but can additionally jump. Our camera orbits around the character. All objects in the scene have colliders (marked static when possible to improve performance). The pickups and rebuildable are all interactive.

Feedback: Collecting the pickups creates a small sparkle in the air, and plays a sound. Using weapons makes a sound. Bringing the items back to the rebuildable plays a rebuild animation and plays sounds.

Win/Fail: Like the 2D levels, the game will show a “You died!” screen when you die to the enemies, allowing you to restart the level. When the rebuildable is fully rebuilt, a flash will play, dialogue will tell the player they have finished the level, and the enemies will all despawn.

Performance: Our lighting is baked and textures are (almost all, except for the glass) GPU instanced to improve performance. Everything in the scene except for the players, enemies, and rebuildables are marked static. Our draw calls hover around 100-200.

Part 1 feedback addressed

1. **“Connection between Theme and Gameplay could be strengthened... environmental message feels disconnected from player actions. Consider adding short messages explaining how each level contributes to climate change”**
 - a. We added dialogue that plays on the completion of each level that explains a little bit more about what exactly the item they rebuilt does for the environment, or how a person can make meaningful climate change in real life.
2. **“Inclusion of audio elements good... expanding on sounds could increase immersion”**
 - a. Added main menu music, sounds for trash lid weapon, footstep sounds, and nature sounds. Also added an audio slider to adjust groups of sounds.
3. **“Home area lacks clear direction. Consider labeling levels or providing clear instructions.”**
 - a. At game start, we now inform the player to head to the forge, then head to the right to go to level one. We added names to all levels. When a level is completed, it tells you where to go next.
4. **“Menu design visually basic, missing return path options.”**
 - a. Added a “Save & Exit to Main Menu” option on the pause menu. Changed the opening animation for the pause menu. Added fade ins/fade outs. Added color to menus; however we felt that our forge menu was already good (it has custom art assets, animations, and updates dynamically.) We also didn’t add an “Exit Game” button on the pause menu, as exiting directly from a game without saving is not an option that many games have (they would normally go back to the main menu, and you can exit from there). We kept the Quit Game button that our main menu had in the last assignment.
5. **“At the start of gameplay, the player receives no guidance or objective clues. Consider adding brief tutorials, pop-up hints, or goal indicators”**
 - a. We added dialogue that tells the player exactly what to do, as well as exactly how many “components” they need to find in the level area to full rebuild the object. We also added dialogue that tells the player more about how to play (press “E” to pickup components, LSHIFT to dash, click / right click to attack.)
6. **“Applying a physics material directly to the player is not ideal...”**
 - a. Removed the physics material from the player.
7. **“While some modular design elements are visible, consider expanding modular systems to include reusable components...”**
 - a. Slightly refactored weapons to make them more modular, allowing you to extend WeaponBase and only implement the physics side of things. Kept the existing IInteractor, IPickup, and IItem architecture. Kept the existing IHealth architecture.

8. **“there are no strong audio or visual cues for key actions such as collecting items, hitting obstacles, and completing or failing a level.”**
 - a. Added particles on item pickup. Added sounds on item pickup. Added flash screen on level completion. Added environment change on level completion. Added sounds on level completion. Did not add sounds for hitting obstacles, as we felt like the player hits obstacles frequently, and having constant “obstacle-hitting” sounds would get a little overwhelming.
9. **“Consider enabling rotation controls (e.g., arrow or A/D keys) to change the player’s direction.”**
 - a. Added a player sprite that changes animation direction based on player movement.
10. **“Include player-centric innovation options like “Juice packs” (camera shake, screen flash, or smooth transitions), Saving and loading player progress, Environmental feedback systems (e.g., pollution decreases as goals are achieved)”**
 - a. Added screen flash when level completes, and smooth fade in / fade out transitions when loading between scenes.
 - b. Expanded on the existing save system from assignment 2. The game now saves your active scene when you save, so reloading the save slot will put you into the same level you saved in. Also added dialogue scenes played to the save data.
 - c. Added environmental feedback: the world becomes completely green and nature sounds start playing as the goal of rebuilding the object completes. *Note: the 3D level does NOT update to show a green world, but nature sounds still play.*
11. **“The font size and colour contrast on the game explanation screen can be improved for readability.”**
 - a. Increased font size to all fonts in the game. Improved font readability across the entire game by adding an outline to text and changing color where contrast was poor.
12. **“Include commentary or feedback text... to help orient players.”**
 - a. Added level completion dialogue to all levels.
13. **“Although sound, volume, and speed adjustment options were included, no audible audio output was detected, suggesting a functional issue.”**
 - a. We double checked this and everything was linked correctly. In fact, playing assignment two on browser ([link](#)), the sounds play as expected (dash sounds, basic melee sounds, purchase / equip sounds, component sounds). However, we added additional sounds to many parts of the game, and added audio sliders in-game to change sound levels of different groups.

Who did what

These are changes made by us since the first assignment. I grouped them by PR to make it a little clearer what changes happened, since some of the biggest assignment 2 improvements aren’t easily visible.

Jonathan	Heath	Dylan	Matthew*
This README. Comparison videos. PR #1 - Abstracting weapon handler into a base class to make creating new weapons simpler PR #2 - Give home area fourth path PR #4, #6 - Adding 3D interaction and outline around interactables (that ended up being unused) PR #5, #33, #41 - 3D Rebuildable code, model, textures, and animations PR #34, #41 - 3D Rebuildable component model, textures, bloom post-processing	PR #16 - Player walking particles and walking sound effects PR #32, #38, #46 - Add terrain to 3D menu. Import a lot of new 3D assets for 3D menu and 3D level. Add objects to 3D menu. Add panning camera around 3D scene. PR #50 - Add music to the main menu. PR #52 - Add nature sounds that should play after the rebuildable is built. PR #56 - Add nature sounds to home area.	PR #29 - 3D Enemy implementation tracking player with NavMeshes. Add lunge attack to 2D enemies to make their attacks more telegraphed. Fix enemy spawner pooling bug. Rework loot system to work based on percentages instead of constant tables. Add 3D enemy spawner. PR #37 - Add 3D enemy lunge. 3D Enemy spawner bugfixes. PR #39 - Player sprites, player animations, forge animation bugfixes,	PR #7 - 3D Player Movement with force, enabling us to make direct changes to the player force with things like weapons or enemies <i>Note: We aren’t sure what happened to Matt. I honestly hope he’s doing okay, since he suddenly vanished without any warning. The last I heard from him was Oct 25th.</i>

PR #9 - Changing 2D weapons to attack by click direction, translation between 2D & 3D weapons PR #9, #11 - 3D melee weapon code, model, textures, animations PR #10 - 3D Ranged weapon code, model, textures, animations PR #13 - Level loader from 3D->2D PR #14, #15, #22, #43 - Dialogue UI, dialogue display code, dialogue triggers, dialogue events PR #22, #24, #43 - Add a “Flash” screen that plays when a level completes. Add a “WorldHealer” that triggers on flash. Add dialogue to levels 1, 2, 3, 4 that plays on enter, walk towards pickup, and level completion and shares significance of the actions taken by the player. PR #17 - Refactor main menu to own scene and change display mode to anchor-based PR #18 - Refactor pause menu and change display mode to anchor-based PR #19 - Removed old ActiveGameManager and moved all functionality to SaveManager, which was renamed to GameManager. Added dialogue scene saving so that dialogue doesn’t replay. Decoupled code as much as possible from the save system. Removed saving/loading on scene exit / enter. Turned player to DontDestroyOnLoad and centralized player data tracking. Remove dependency GameManager had on LevelManager and made it so that GameManager functions independently. Add variable player spawnpoint based on which scene you enter from. PR #20 - Add fade in / fade out transitions on scene changes PR #21, #30 - Add a FreezeEntitiesManager PR #27 - Decouple player from Forge purchasing, refactor		spawner bugfixes, freezing 2d enemies when they should be frozen PR #53, #54 - 3D loot items dropped by enemies PR #31 - Bugfix (Enemy attack null) PR #58 - Bugfix (Level 4 spawner locations & 2D enemy collisions)	
---	--	---	--

forge to run on purchase streams PR #40 - Add component pickup UI that displays current objects held in the player's inventory. PR #42 - Add a 3D camera controller with a combination of scripts and Cinemachine PR #44 - Pickup particles that spawn when picking up items. Refactored existing DustPool particle pool to work with any particle system. Converted the Dash particles to a pool. Made pickup particles a pool. PR #47 - Add TUNED_VARIANTS easy/hard difficulty scriptableobjects for Enemies, Player movement, and player health. Refactor existing code to allow us to drag in those scriptable objects. PR #48 - Add audio sliders and audio groups to menus. Add audio values to save system. PR #49 - Build level 4 PR #8 - Bug fixing (loading) PR #26 - Bug fixing (Currency ui not being synced between scenes) PR #35 - Bug fixing & accessibility improvements (Forge & Text) PR #36 - Bug fixing & accessibility improvements (Dialogue & Path text) PR #51 - Bug fixing (many small changes) PR #55 - Bug fixing (two major fixes around 3D level loading, and many small changes) PR #57 - Bug fixing (freeze screen & weapons) PR #59 - Bug fixing (pause screen & spawners) PR #45 - Accessibility improvements by adding how-to-play tips to second dialogue. PR #12 - Physics allocation optimizations			
--	--	--	--

Known issues

- At smaller resolutions, the currency inventory text will start to overlap with nearby sprites.
- At larger resolutions, the component inventory UI will be very small

- Entities (player and enemies) will still be animating when frozen
- In the WebGL build specifically, enemies will sometimes not spawn in level 4 (the 3D level). We have no idea what is causing this.
- In the WebGL build specifically, text and UI will shrink. I tried debugging this but nothing I did fixed the problems. This makes everything a lot smaller than it should be, reducing the overall accessibility of the game.

Where to find comparison videos / images

On github: **Comparisons**/*. We did not include comparisons for absolutely everything we changed, as there is a lot, but we tried to capture as many of the big-ticket items as we could.

Third-party assets

Water level sprites: <https://ninjikin.itch.io/water> and <https://beeler.itch.io/top-down-earth-tileset>

UI: <https://github.com/Brackeys/Health-Bar/tree/master/Health%20Bar/Assets/Sprites> and <https://last-tick.itch.io/pixel-bags>

Font: <https://assetstore.unity.com/packages/2d/fonts/free-pixel-font-thaleah-140059>

Ruins: <https://opengameart.org/content/ancient-ruins-pixel-art>

Tent: <https://opengameart.org/content/tent-3>

Nature assets: <https://pixelfrog-assets.itch.io/tiny-swords>

Blacksmith sprite: <https://opengameart.org/content/lpc-blacksmith>

Background palette: <https://pixelfrog-assets.itch.io/tiny-swords>

2D Character sprite: <https://opengameart.org/content/zelda-like-tilesets-and-sprites>

Desert assets: <https://assetstore.unity.com/packages/3d/environments/landscapes/pf-desert-pack-free-3d-psx-retrostyle-304703>

Trash can model: <https://assetstore.unity.com/packages/3d/props/trash-bag-trash-box-316725>

Tree trunk: <https://assetstore.unity.com/packages/3d/vegetation/trees/cracked-tree-trunk-277007>

References (For statements made in dialogue about climate action and effectiveness)

Littering: <https://www.environmentalvolunteers.org/effects-of-littering/#littering>

Windmills: <https://www.energy.gov/eere/wind/advantages-and-challenges-wind-energy>

Solar Panels: <https://www.eia.gov/energyexplained/solar/solar-energy-and-the-environment.php>

Global CO2 levels:

<https://www.climate.gov/news-features/understanding-climate/climate-change-atmospheric-carbon-dioxide>

Mechanical trees: <https://netzerocompare.com/academy/exploring-the-potential-of-mechanical-trees>